Permutations

AlgoExpert · Medium · Recursion

The problem

- All orderings of a list of distinct ints (empty input yields an empty list).

Intuition

- Build permutations by swapping each remaining element into the current position.

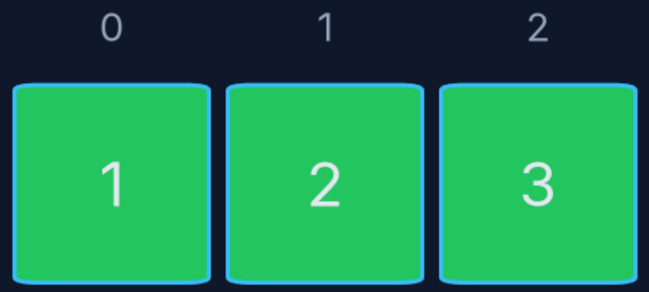
Brute force vs optimal

- Build by inserting into every position → $O(n! \cdot n^2)$
- Swap-based backtracking → $O(n! \cdot n)$
- Output size n! is the true lower bound

Approach

- 1. Recurse on index i
- 2. Swap each later element into position i
- 3. Recurse, then swap back

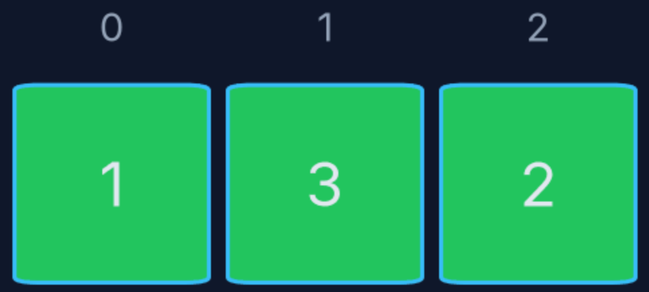
Worked example




```
permutations: [[1, 2, 3]]
```

Leaf → record [1, 2, 3]

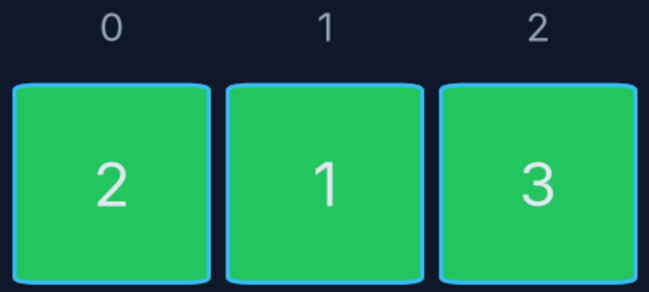
Worked example




```
permutations: [[1, 2, 3], [1, 3, 2]]
```

Leaf → record [1, 3, 2]

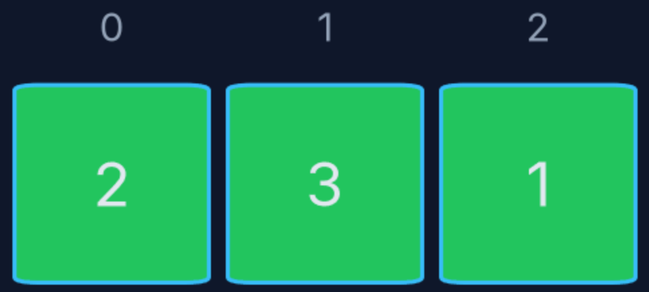
Worked example




```
permutations: [[1, 2, 3], [1, 3, 2], [2, 1, 3]]
```

Leaf → record [2, 1, 3]

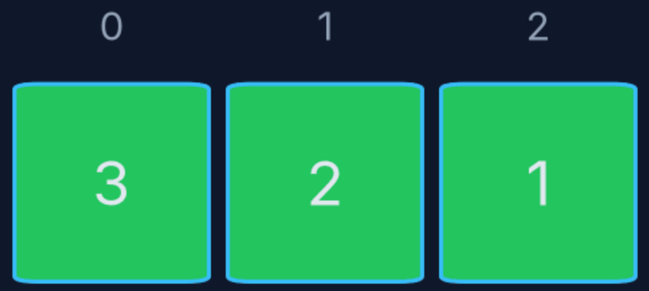
Worked example




```
permutations: [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1]]
```

Leaf → record [2, 3, 1]

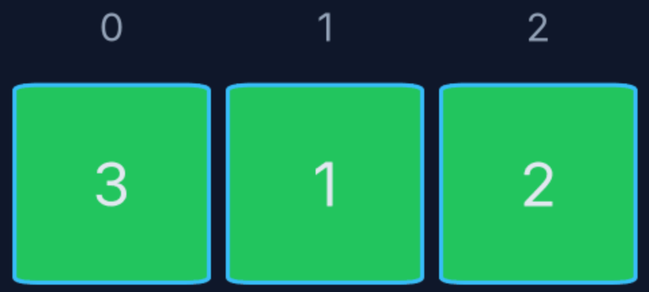
Worked example




```
permutations: [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 2, 1]]
```

Leaf → record [3, 2, 1]

Worked example




```
permutations: [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 2, 1], [3, 1, 2]
```

Leaf → record [3, 1, 2]

Java solution

```java
public class Permutations {
    public static List<List<Integer>> getPermutations(List<Integer> array) {
        List<List<Integer>> out = new ArrayList<>();
        if (!array.isEmpty()) permute(0, new ArrayList<>(array), out);
        return out;
    }

    private static void permute(int i, List<Integer> a, List<List<Integer>> out) {
        if (i == a.size() - 1) { out.add(new ArrayList<>(a)); return; }
        for (int j = i; j < a.size(); j++) { Collections.swap(a, i, j); permute(i + 1, a, out); Collections.swap(a, i, j); }
    }
}
```

Complexity

- Time: O(n * n!)
- Space: O(n * n!)

Edge cases & pitfalls

- Empty input → []
- Undo the swap after recursing
- Copy the array when recording

Interview tips & follow-ups

- Follow-up: duplicates → skip repeated values per position
- Next permutation (lexicographic)

Recap

- Build permutations by swapping each remaining element into the current position.
- Complexity: O(n * n!)